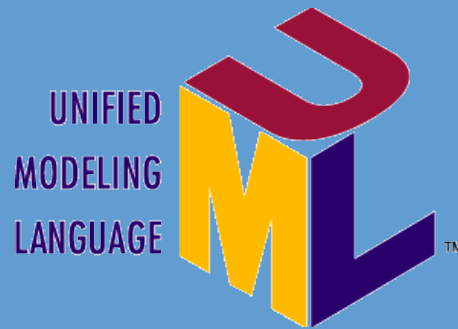


Diagramme de classes

Diagramme de séquence



Katy SAINTIN

Janvier 2016

Le langage de modélisation unifié, de l'anglais *Unified Modeling Language* (UML), est un **langage** de modélisation graphique normalisée à base de pictogrammes conçu pour visualiser la conception d'un système. Il est couramment utilisé en développement logiciel et en conception orientée objet.

Expriment de manière générale la structure statique d'un système.

La classe regroupe un ensemble d'objets ayant les mêmes propriétés et les mêmes comportements.

Les propriétés déterminent les états possibles de l'objet.

Les généralités sont contenues dans la classe et les particularités dans les objets.

Pokemon

-nom : String
-type : **enum** {Normal, Feu, Eau}
-pc : **int**
-favori : **boolean**

+recharger(**int** pc) : **void**
+attaquer() : **int**

Représentation graphique

- + : Élément publique (public)
- # : Élément protégé (protected)
- : Élément privé (private)

```
package fr.ifadelorozoy;  
  
public class Pokemon {  
  
    public static enum Type {Normal, Feu, Eau};  
    private String nom;  
    private Type type = Type.Normal;  
    private int pc;  
  
    public void recharger(int pc){  
        this.pc = this.pc + pc;  
    }  
  
    public int attaquer(){  
        return 0;  
    }  
}
```

Implémentation Java

<<interface>>

IPokemon

+Type : **enum** {Normal, Feu, Eau}

+recharger(**int** pc) : **void**

+attaquer() : **int**

Représentation graphique

```
package fr.ifadelorozoy;

public interface IPokemon {

    public static enum Type {Normal, Feu, Eau};

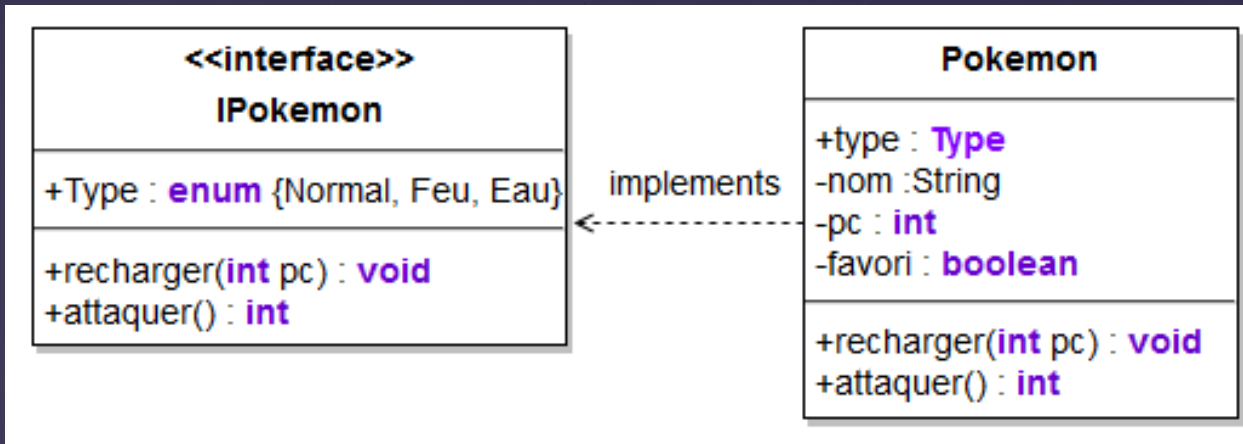
    public void recharger (int pc);

    public int attaquer();

}
```

Implémentation Java

L'implémentation d'un interface



Représentation graphique

```
package fr.ifadelorozoy;

public class Pokemon implements IPokemon {

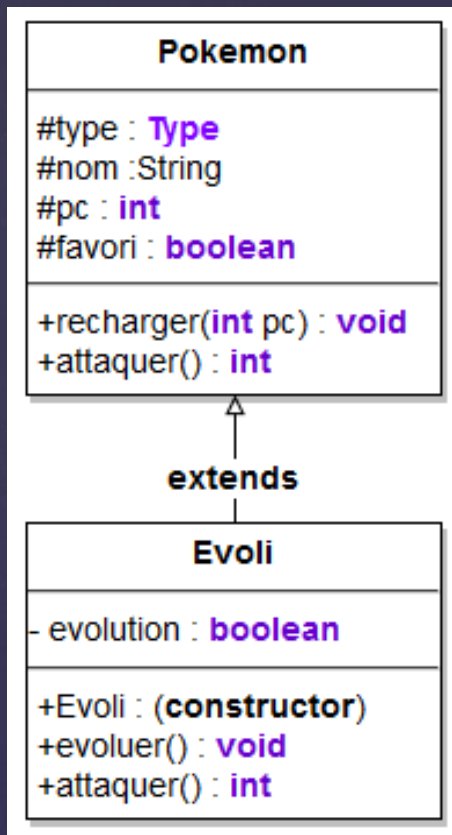
    private String nom;
    private Type type = Type.Normal;
    private int pc;
    private boolean favori = false;

    public void recharger(int pc){
        this.pc = this.pc + pc;
    }

    public int attaquer(){
        return 0;
    }

}
```

Implémentation Java



Représentation
graphique

```

package fr.ifadelorozoy;

public class Evoli extends Pokemon {

    private boolean evolution = true ; //Ce pokemon peut evoluer

    //Surcharge du constructeur
    public Evoli() {
        super(); //Appel le constructeur parent
        nom = "Evoli"; //Initialise le nom
    }

    //Surcharge de la methode attaquer
    @Override
    public int attaquer() {
        System.out.println("Attaque evoli");
        return super.attaquer(); //Appel de la methode parent
    }

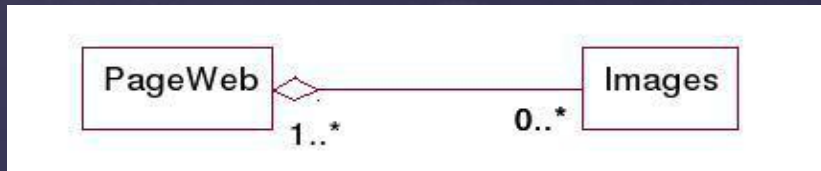
    public void evoluer(){
        //Code pour faire evoluer Evoli
        nom = "Aquali";
        this.pc = this.pc + 100;
        evolution = false ; //Ce pokemon ne peut plus evoluer
    }
}
    
```

Implémentation Java

L'association : relation 1-1 avec définition de **rôles** (ex : conducteur)



L'agrégation : notion de "faire partie" de quelque chose avec définition de la **multicplité** (nombre d'objets)



1 : un et un seul

0 .. 1 : zéro ou un

M .. N : de M à N (entiers connus)

* ou 0 .. * : de zéro à plusieurs

1 .. * : de 1 à plusieurs

La composition : contrainte forte entre composite et composant (création, destruction). La multiplicité du côté du composite ne peut prendre que la valeur 1

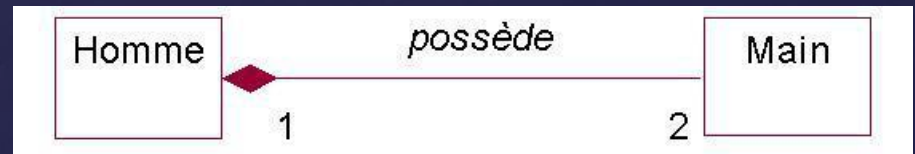
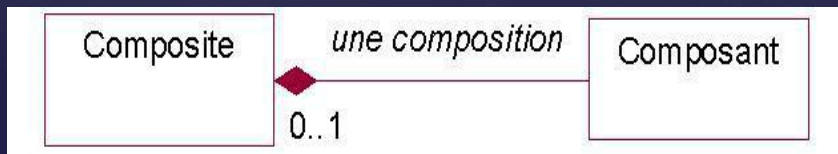
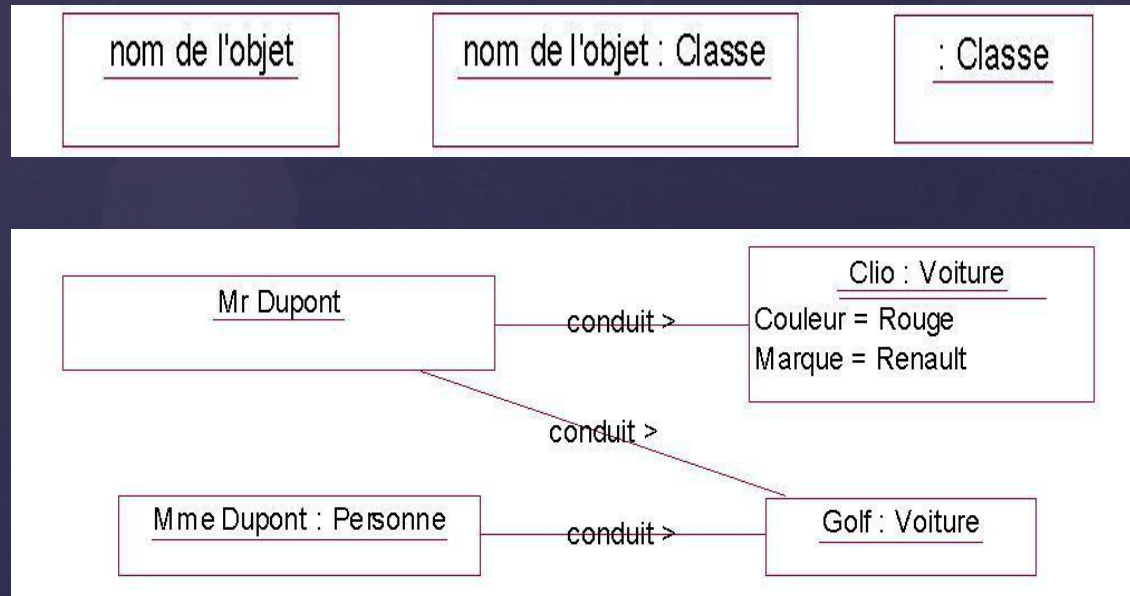
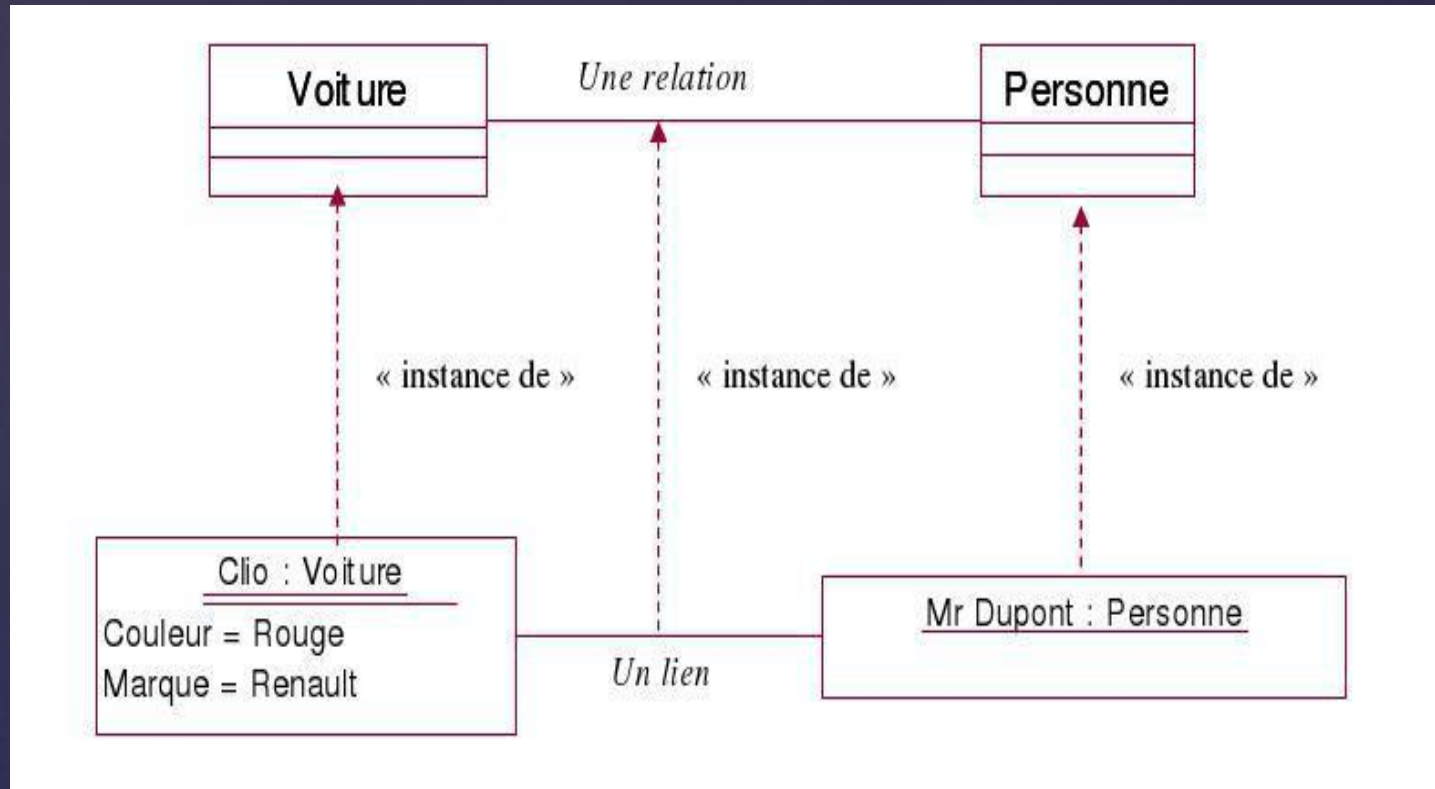


Illustration du diagramme de classes à un instant donné.
S'utilisent pour montrer un contexte (et pour aider au choix des mul-tiplicités)





Représentation graphique des **interactions** entre les acteurs et le système selon un ordre chronologique

